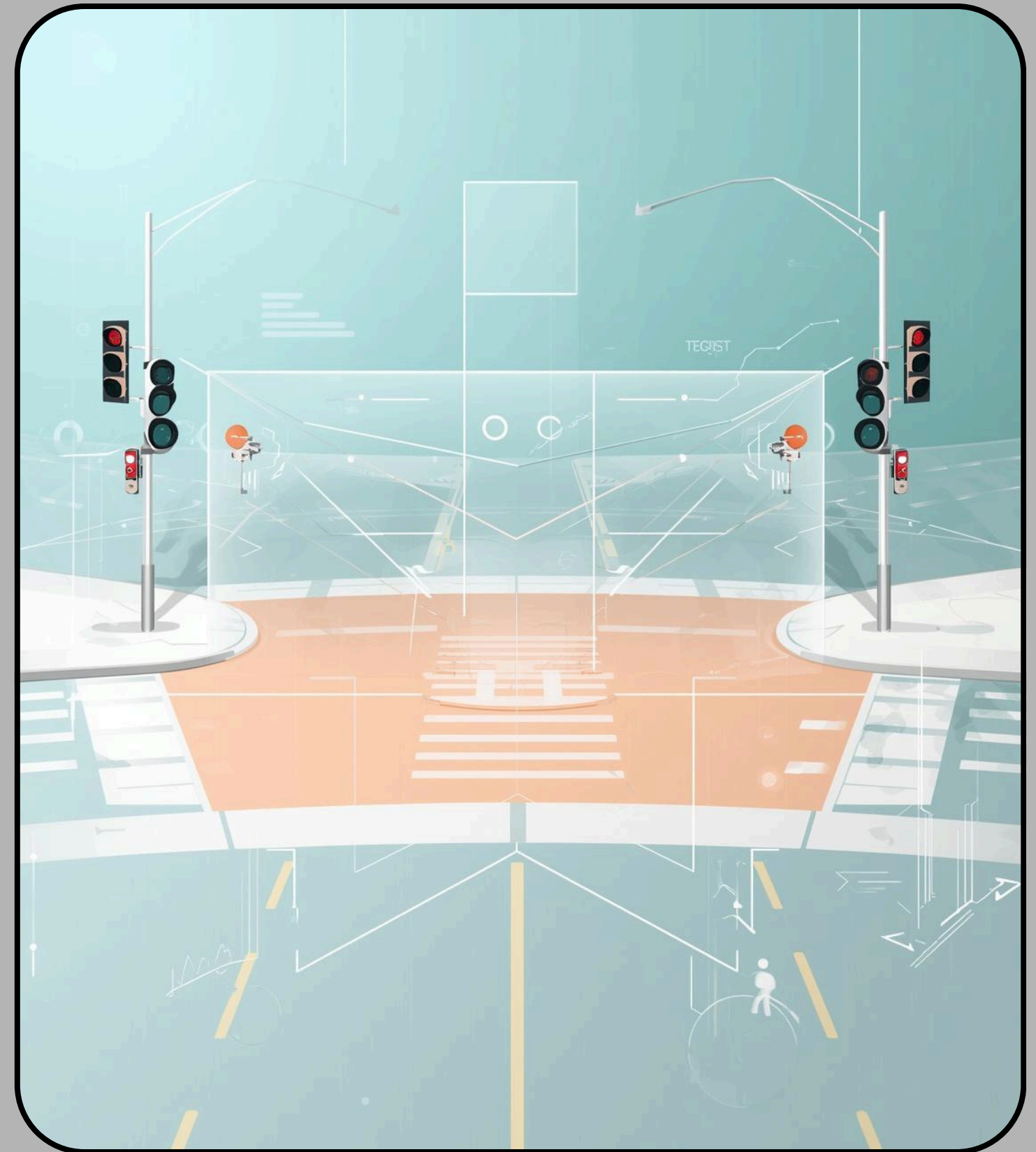


AIOT

# Smart Intersection Solutions

smaRRRt

B1228011 HENGTING LIU  
B1228021 TINGYU CHIU



# Project Overview

- Core Goal:
  - Build a smart traffic light system for pedestrian safety.
- Main Idea:
  - Use YOLOv8 to detect pedestrians and estimate their walking speed.
  - Use SUMO to simulate the traffic intersection.
  - Use TraCI to control the traffic light in real time.

# Project Overview

- How It Works:
  - a. The camera or video detects pedestrians.
  - b. The system calculates pedestrian speed.
  - c. If a pedestrian is too slow, the system sends an event to SUMO.
  - d. SUMO extends the pedestrian green light.
- Expected Benefit:
  - Reduce crossing risk and improve pedestrian safety.

# System Architecture

## 1. YOLO Detection Server

- Detects pedestrians from video frames
- Tracks each pedestrian with ByteTrack
- Estimates walking speed
- Sends slow pedestrian events by HTTP

## 2. SUMO Controller

- Loads the real intersection map
- Runs traffic and pedestrian simulation
- Receives YOLO events
- Uses TraCI to extend the green light

## 3. Web Dashboard

## Perception Layer

- Collect traffic images.
- Detect pedestrians using YOLOv8.
- Visualize results on a monitoring dashboard.

## Communication Layer

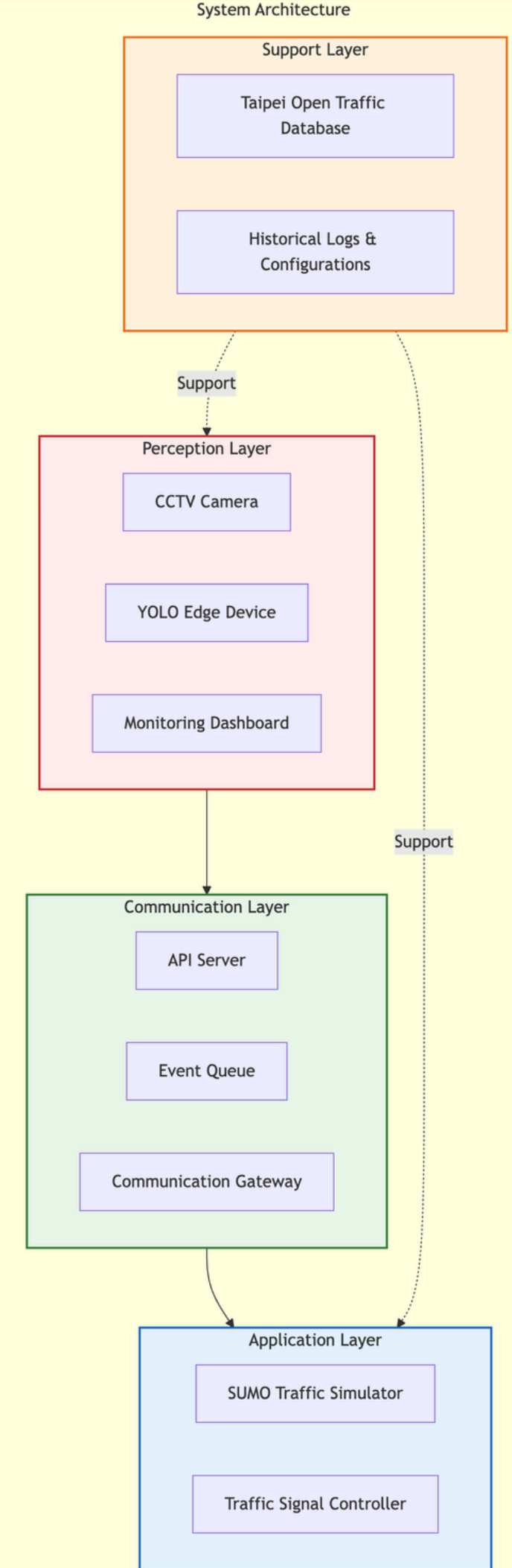
- Receive detection results through APIs.
- Transfer event messages using an event queue.
- Enable communication between system modules.

## Application Layer

- Simulate traffic conditions with SUMO.
- Adjust signal timing according to detection results.

## Support Layer

- Provide traffic data, historical logs, and configuration parameters.
- Support system operation and decision-making.



# Data Flow

Video

→ YOLOv8

→ Speed Estimation

→ HTTP API

→ TraCI

→ SUMO Traffic Light

# Current Progress:

## YOLO Perception Module

| Item              | Old Version         | Current Version                 |
|-------------------|---------------------|---------------------------------|
| Detection Model   | YOLOv8n             | YOLOv8s                         |
| Tracking Method   | DeepSORT            | ByteTrack                       |
| Speed Calculation | Pixel movement only | Homography + EMA smoothing      |
| Processing Design | Multi-thread queue  | Single process loop             |
| Output            | Detection only      | Detection + speed + risk status |

# Why We Changed the Tracking Method?

## Problem with DeepSORT:

- Pedestrians are very small in far-distance camera views.
- Appearance features are not stable.
- The same person may get many different IDs.
- Tracking may drift to empty areas.
- This makes speed calculation unreliable.

## Why ByteTrack Works Better:

- It uses bounding box overlap to match people.
- It does not depend on appearance features.
- It gives more stable tracking IDs.
- It is better for small and far pedestrians.
- It helps the system calculate speed more correctly.

# Speed Estimation and Risk Levels

- **Speed Estimation Method:**
  - Convert image position to real-world position
  - Use Homography to reduce perspective error
  - Calculate movement distance over time
  - Use EMA smoothing to reduce speed shaking
- **Risk Levels:**
  - Danger: speed  $< 0.60$  m/s
  - Slow:  $0.60$  to  $0.90$  m/s
  - Normal: speed  $\geq 0.90$  m/s
- **Trigger Rule:**
  - If the pedestrian is Slow or Danger, the system sends an event to SUMO.

# Current Progress:

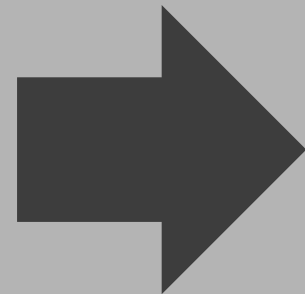
## System Control & Automation

- **API Bridge Completed:**

- We built an HTTP API between YOLO and SUMO.

- **YOLO Sends:**

- Intersection ID
- Camera ID
- Track ID
- Average speed
- Slow pedestrian status
- Crossing edge
- Event TTL



- **SUMO Receives:**

- The event is stored in a queue
- The system checks if the event is valid
- The system waits for the correct pedestrian green phase
- When the phase starts, TraCI extends the green light

- **Result:**

- YOLO and SUMO can now communicate successfully.

# Current System Flow

- **Step 1: Load Simulation**
  - SUMO loads the road map, vehicles, pedestrians, and traffic lights.
- **Step 2: Apply Real Timing**
  - TraCI applies the official traffic light timing plan.
- **Step 3: Detect Pedestrians**
  - YOLOv8 detects pedestrians and ByteTrack tracks them.
- **Step 4: Estimate Speed**
  - The system calculates each pedestrian's walking speed.
- **Step 5: Send Event**
  - If the pedestrian is slow, YOLO sends an HTTP POST event.
- **Step 6: Extend Green Light**
  - When the correct pedestrian phase arrives, TraCI extends the green light by 10 seconds.
- **Step 7: Save Data**
  - The system records speed data and traffic light extension logs.

# Test Result:

## Green Light Extension

- **End-to-End Test Result:**
  - YOLO → HTTP API → TraCI → SUMO → Green Light Extended
- **Successful Results:**
  - The HTTP API received slow pedestrian events.
  - SUMO returned the response: {'status': 'ok'}
  - The event was matched to the correct phase.
  - Phase 5 was extended by 10 seconds.
  - The event log recorded the extension result.
- **Meaning:**
  - The main control function works successfully.

**DEMO**

# References and Acknowledgements

- <https://www.airitilibrary.com/Article/Detail/10272275-202009-202010080005-202010080005-321-342>
- <https://www.aimspress.com/article/doi/10.3934/mbe.2024077>
- <https://www.bohrium.com/paper-details/research-on-pedestrian-detection-and-deepsort-tracking-in-front-of-intelligent-vehicle-based-on-deep-learning/817361004055756800-3177>
- <https://ijca.phdfocus.com/archives/volume183/issue27/32097-2021921653/>
- <https://futurecity.cw.com.tw/article/3661>
- <https://join.gov.tw/idea/detail/84c4c73f-241a-40bf-9fbe-155226079336>
- <https://roadsafety.tw/Dashboard/Custom?type=統計快覽圖表>
- [道路交通標誌標線號誌設置規則](#)